

Relevamiento y justificación

El hardware necesario para el correcto desarrollo así como mantenimiento, escalabilidad y disponibilidad del sistema a realizar, implica que se almacene y acceda a datos sensibles de cientos de miles de usuarios rápidamente y sin retrasos. La configuración de hardware recomendada en cuanto a una instancia virtualizada para el servidor del hospital serían 16gb de ram, 4 vCPUs, 200gb de almacenamiento en disco SSD, y que el rendimiento de la red sea de 1 Gbps o superior, lo cual permitiría mantener la documentación de cientos de miles de usuarios sin problema por muchos años.

Teniendo tal configuración, el servidor Amazon EC2 cuesta aproximadamente 110 dólares al mes. Los servidores del hospital deberían tener ello a su alcance, a diferencia del servidor que consigue el equipo de desarrollo para demostrar con datos ficticios el funcionamiento, siendo adecuados 4gb de ram, 2 vCPUs y 50 gb de almacenamiento en estado sólido, esto debido a que no se procesarán funciones que exijan potencia de cálculo bruta, y se simulará con datos a menor escala.

La lectura y escritura de registros médicos consume poca potencia de cálculo, así que es suficiente hardware para realizar dichas simulaciones. El costo de un VPS con las características recién mencionadas, se encuentra a 25 usd en Amazon y a 32 usd en Netuy como un par de opciones disponibles. Es importante nombrar que dichos requisitos de hardware son los adecuados para demostrar el correcto funcionamiento de ambos sistemas siendo realizado por estudiantes, luego será cargado en el servidor del hospital el cuál cuenta con hardware más robusto.

De todas formas se opta por utilizar un equipo propio para hospedar el servidor de manera local mediante VirtualBox, específicamente la versión 7.2.6 que fue la que funciona adecuadamente con CentOS 10 a diferencia de la versión 7.2.8 que se han encontrado inconvenientes. Se utilizarán las características de hardware modestas ya mencionadas, para así reducir costos y tener la ventaja de una latencia mínima. Cabe mencionar que no se utilizará bare metal ya que no vemos necesario trabajar directamente en el hardware, la virtualización permite optimizar recursos y

modificarlos mediante configuración. Se es consciente que esto conlleva capas extra en el sistema, lo cual también aísla el mismo para que en caso de fallar no se afecten otros datos críticos.

Sistema operativo y soporte técnico

El sistema operativo a utilizar será CentOS Stream 10 ya que ofrece un punto intermedio entre estabilidad y software actualizado, el cuál permite un desarrollo de forma muy vinculada a RHEL (Red Hat Enterprise Linux).

La recomendación de soporte técnico es ofrecida por TuxCare, el servicio llamado KernelCare Enterprise Linux que brinda parches y actualizaciones en todo momento sin necesidad de reiniciar el servidor, ofreciendo soporte del ecosistema que utiliza CentOS a un precio que puede rondar los 60 dólares anuales. Es necesario completar los datos y realizar la consulta con la empresa para obtener un presupuesto actualizado, el HDC se hace responsable y maneja los criterios de su contratación.

<https://tuxcare.com/enterprise-live-patching-services/kernelcare-enterprise/>

Se aclara que el Hospital de Clínicas ya cuenta con su propio soporte técnico contratado, motivo por el cuál no va a ser necesaria la compra del servicio de TuxCare. De todas formas se lo menciona como posible consideración.

La distribución utilizada garantiza actualizaciones de seguridad y soporte las cuales culminan el primero de enero de 2030, asegurando la confiabilidad de este proyecto incluso luego del tiempo que debe mantenerse funcional para el egreso de los estudiantes, reforzando el compromiso con el proyecto.

Herramientas y funciones nuevas para los desarrolladores

Al ser una versión oficial nueva, RHEL10 incorpora actualizaciones importantes para permitir a los desarrolladores trabajar de forma adecuada, entre ellas se encuentran varias directamente relacionadas con herramientas del proyecto como:

PHP 8.3: Implementa compatibilidad con el algoritmo de contraseña Argon 2, se gestionan los errores y excepciones con clases modernas e implementa paquetes para usar distintas bases de datos.

Git 2.4.7: Resulta más eficiente gracias al formato reftable mejorando la gestión y capacidad de ajuste en cuanto a las referencias, mapas de bits y rendimiento al manipular índices incrementales de múltiples paquetes para actualizaciones óptimas.

MySQL 8.4: Mejora el control de contraseñas de administradores, además de ser más seguro y eficiente en cuanto a la autenticación y backups con anteriores versiones de MySQL.

No se utilizan exactamente esas versiones de PHP ni MySQL en el entorno de desarrollo ya que aún no son implementadas de forma estable en el paquete XAMPP, pero hay completa compatibilidad entre ellas, se utilizan PHP 8.2.12 y MariaDB 10.4.32 el cual su base es MySQL. Si se cuenta con una de las últimas versiones de Git, la 2.52.0 (o 2.54.0 en Windows, ambas compatibles y sin funcionalidades diferentes que nos impidan el uso de cualquiera de las dos), siendo más que suficiente para este sistema y ya probada en entornos empresariales.

Requerimientos mínimos/compatibles del PC Cliente

La PC utilizada por los funcionarios administrativos debería ser compatible con el programa mientras se cumplan los puntos en base a hardware y software mencionados a continuación:

Siempre y cuando cuente con un CPU (procesador) de 2010/2012 en adelante con arquitectura x64 bits y al menos 2 núcleos, el cuál debe soportar los motores de

navegación modernos, siendo compatible nativamente con navegadores web como Google Chrome, Microsoft Edge, Mozilla Firefox, Opera, o similares.

Los navegadores dejaron de dar soporte y actualizaciones para procesadores de 32 bits, por ejemplo Chrome eliminó oficialmente su instalador de dicha arquitectura. Algunos ejemplos de CPUs que se podrían utilizar serían Intel Core i3 2100 / 2130, o Intel Core i5 2400 / 2500 con Intel HD Graphics respectivo de cada modelo. Los procesadores mencionados son compatibles con los SO windows 10 y linux con kernel 5.4 en adelante.

La memoria ram mínima necesaria son 8GB DDR3, siendo lo primordial a día de hoy para un equipo de oficina y que soporte un sistema operativo oficial, pudiendo navegar y trabajar en la aplicación. También se debe contar con el 20% de espacio disponible en un disco SSD de 256GB, para no ralentizar el equipo, así como soportar la memoria caché del navegador, cookies y almacenamiento local. Estos discos ofrecen una velocidad muy superior a un disco duro tradicional (HDD). Cualquier tarjeta de red común o wifi es suficiente para la conectividad.

Como sistema operativo el PC cliente debe contar mínimo con Windows 10 de 64 bits, o Linux con kernel 5.4 en adelante priorizado debido a los equipos con los que ya cuenta el HDC.

Cabe destacar que se toman en cuenta los requerimientos mínimos en caso de querer correr de forma oficial el software a utilizar, pero es un equipo que al día de hoy quedaría obsoleto y puede ralentizar el trabajo de los administradores. En caso de utilizar Windows 10, se advierte que está por quedarse sin soporte y por ende sin actualizaciones de seguridad en Octubre de 2026.

Requerimientos recomendados del PC Cliente

A continuación se mencionan los requisitos recomendados del PC Cliente, para poder utilizar la aplicación de forma ideal, fluida, completamente actualizada y compatible con las tecnologías necesarias, sin que la PC sea un inconveniente o

impida el correcto desempeño en el trabajo por parte de los funcionarios administrativos del Hospital de Clínicas.

- **Procesador:** Intel Core i5 de octava generación (en adelante) o Ryzen 5 2600g/3400g o superior, (con gráficos integrados).

- **Ram:** 8GB DDR4

- **Almacenamiento:** Disco de 256GB NVMe m.2.

- **Sistema Operativo:** Cualquier distribución moderna de Linux compatible con las tecnologías utilizadas, el HDC cuenta con distribuciones compatibles en su . La alternativa es Windows 11.

- **Fuente de poder:** Se recomienda una PSU Certificada al menos 80 plus bronze para la eficiencia y protección de los componentes, te da una garantía de seguridad, en cambio una fuente genérica no es lo más adecuado para el equipo porque aumenta el desgaste de los componentes y hace trabajar más el VRM de la placa madre, ya que la energía puede llegar en picos más disparejos, lo cuál afecta y reduce su vida útil considerablemente.

- **UPS:** Como medida extra de seguridad en caso de un corte en la electricidad, este componente almacena energía de reserva en su interior, lo que permite seguir con el PC encendido durante un mayor margen de tiempo, dando garantía para terminar y resguardar el trabajo pendiente que se estaba realizando, ya que puede contener información importante de un paciente del Hospital o de la logística, por ende no se puede perder o corromper bajo ningún concepto. Así que tener este componente da un tiempo para organizarse y resolver lo que se estaba haciendo, para luego apagar el equipo de forma segura sin afectar sus componentes, resultando indispensable en el manejo de información crítica del HDC. Gracias al relevamiento de información en la entrevista se sabe que el Hospital cuenta con esta protección para todos sus equipos.

- **Conexión a Internet:** Se recomienda mediante Ethernet a 1000 Mbps, evitar usar WiFi ayuda a una conexión más estable y sin cortes, asegurando la actualización de datos en tiempo real.

Con esas características se garantiza que los funcionarios administrativos van a poder ejecutar todas sus tareas, cargar datos y aplicar actualizaciones en el sistema, evitando inconvenientes al gestionar el ingreso de datos críticos, como los diagnósticos de pacientes o la logística institucional. Esto incluye el monitoreo de vehículos, la correcta gestión de rutas, el aviso de disponibilidad en el HDC e incluso la notificación de cómo van los traslados en proceso, entre otras consultas pertinentes

Paquetes necesarios del sistema

- Generalmente preinstalados

Ciertos paquetes ya vienen preinstalados pero se verificará de todas formas que se encuentren disponibles mediante los comandos en consola, tales como `iproute` ya por defecto de Red Hat para la configuración de red, por ejemplo de rutas estáticas y puertas de enlace, siendo crucial para determinados pasos al conectar el servidor de forma remota junto con SSH haciendo al servidor accesible desde la red del hospital.

Firewalld sirve para controlar qué puertos se mantienen abiertos o cerrados, evitando conexiones indeseadas; `rsyslog` es crucial para monitorear activamente en archivos log lo que ocurre en el servidor, permitiendo corregir errores y auditar accesos, siendo clave para la seguridad considerando que se manejan datos médicos. `Rsync` sirve para transferir archivos entre servidores y así tener respaldos remotos de datos del sistema.

Se utilizará OpenSSH ya que es la principal herramienta y un estándar para la conectividad remota mediante el protocolo SSH, sirve para conectarse al servidor desde otro equipo y es clave para que se pueda modificar el servidor sin necesidad de que el personal tenga que trasladarse al lugar físico donde va a estar el mismo

en el piso 6 del HDC. Ofrece seguridad y confiabilidad, encripta el tráfico para evitar el espionaje, el secuestro de la sesión (connection hijacked) y otros tipos de ataques. Cuenta con múltiples métodos de autenticación y es ampliamente configurable.

Además usaremos el paquete `dnf-plugins-core` para añadir de forma segura repositorios externos y poder administrarlos por el config-manager. Sirve para indicarle al SO donde se encuentra el complemento a instalar, mediante el enlace de la web oficial, por ejemplo para el Docker Engine se le indica y luego también para el plugin Compose, que facilita la administración de servicios, redes y volúmenes en un archivo de configuración YAML. Esto sirve para que luego con un solo comando desde un PC Cliente se puedan iniciar todos los servicios del servidor.

- Instalados manualmente

Se instalará el paquete Docker Engine para la organización en contenedores de código abierto y así construir las aplicaciones, específicamente dentro de Docker estarán plugins de Mysql, Apache y PHP, lo cuál ayuda a separar en distintas partes los procesos del sistema, ayudando a aprovechar la infraestructura y también a que la instalación del sistema ya completo se pueda hacer de forma rápida y simple.

Cuenta con una buena seguridad y ofrece la posibilidad de realizar implementaciones en poco tiempo, así como llevar control de versiones y distribución.

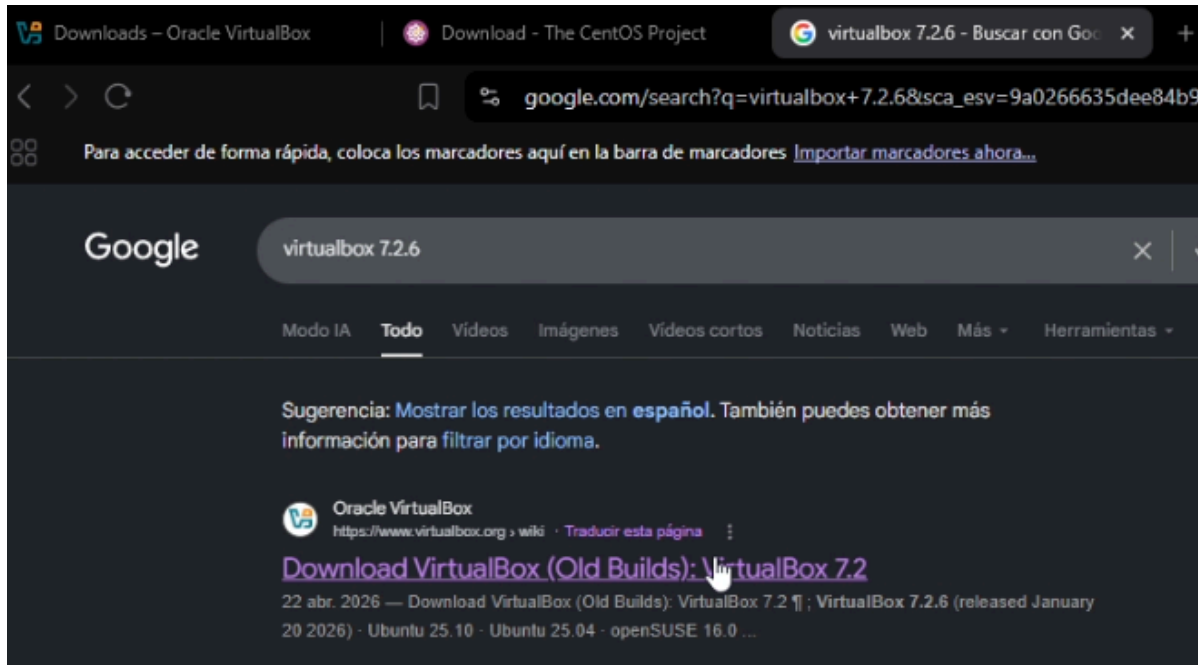
El Docker Engine completo consta de tres paquetes, el `docker-ce` que sería el motor de contenedores, el `docker-ce-cli` que integra la interfaz de línea de comandos para configurar docker desde allí, `containerd.io` para gestionar el ciclo de vida de dichos elementos, y por último el plugin `docker-compose-plugin` para el ya mencionado archivo de configuración YAML.

El paquete git será instalado para poder clonar el repositorio completo de GitHub directamente en el servidor y así desplegar la aplicación, se trata de un sistema de control de versiones distribuido.

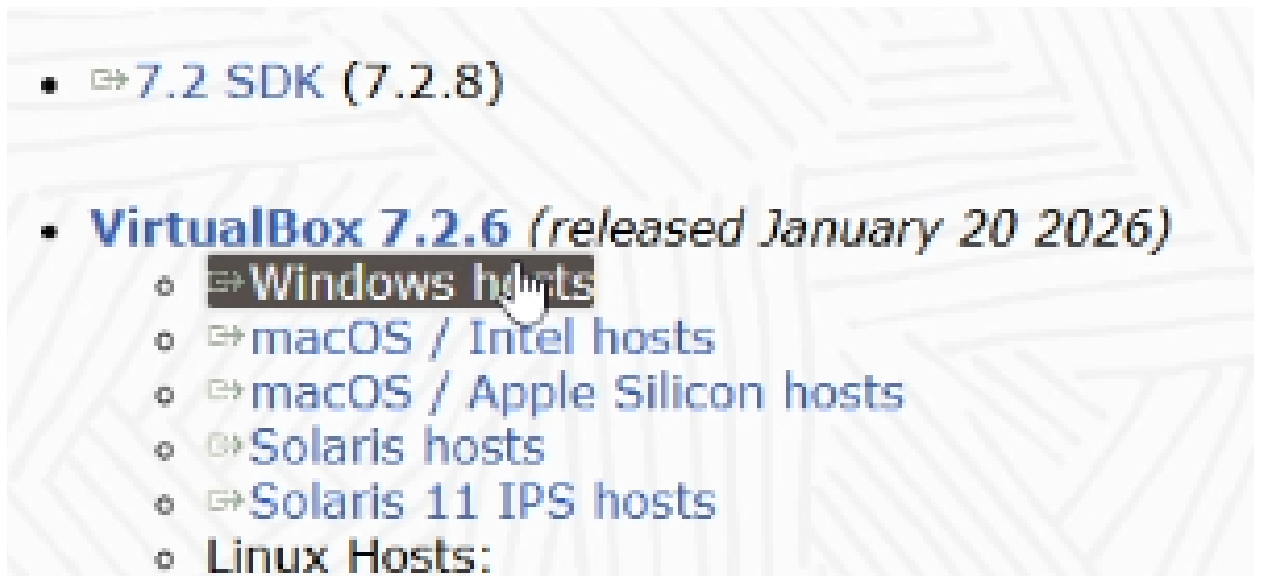
Manual de instalación:

Paso a paso para la instalación del servidor con los paquetes mencionados y configuraciones:

1) Buscar virtualbox 7.2.6 e ingresar al link indicado



2) Descargar e instalar virtualbox 7.2.6a



Oracle VirtualBox 7.2.6 Installer



Ready to Install

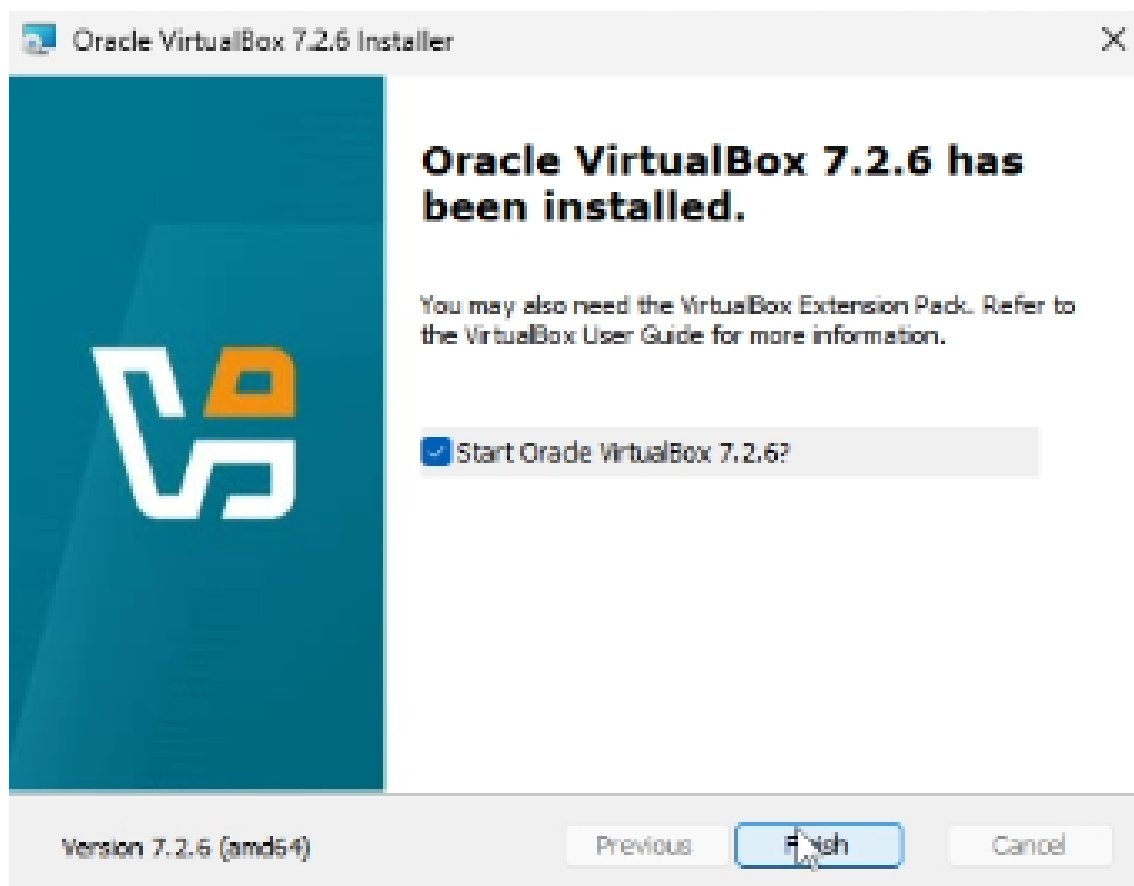
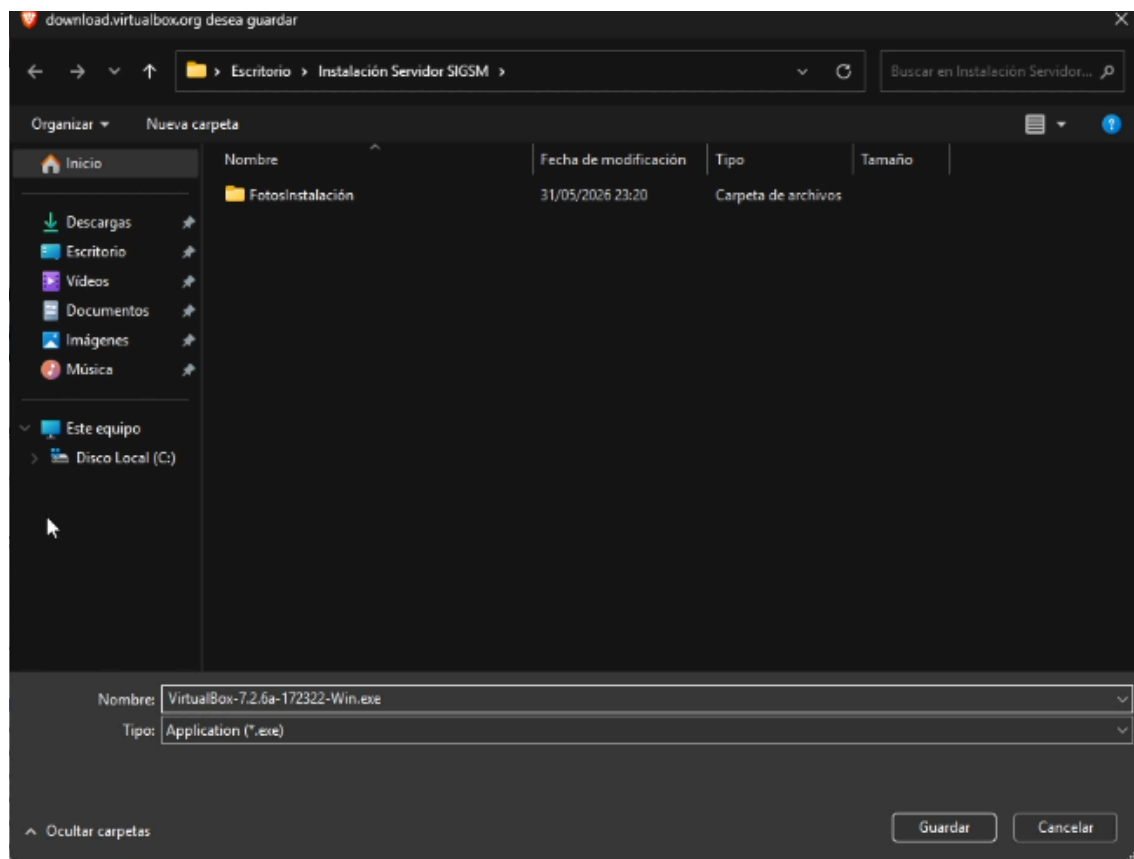


Version 7.2.6 (amd64)

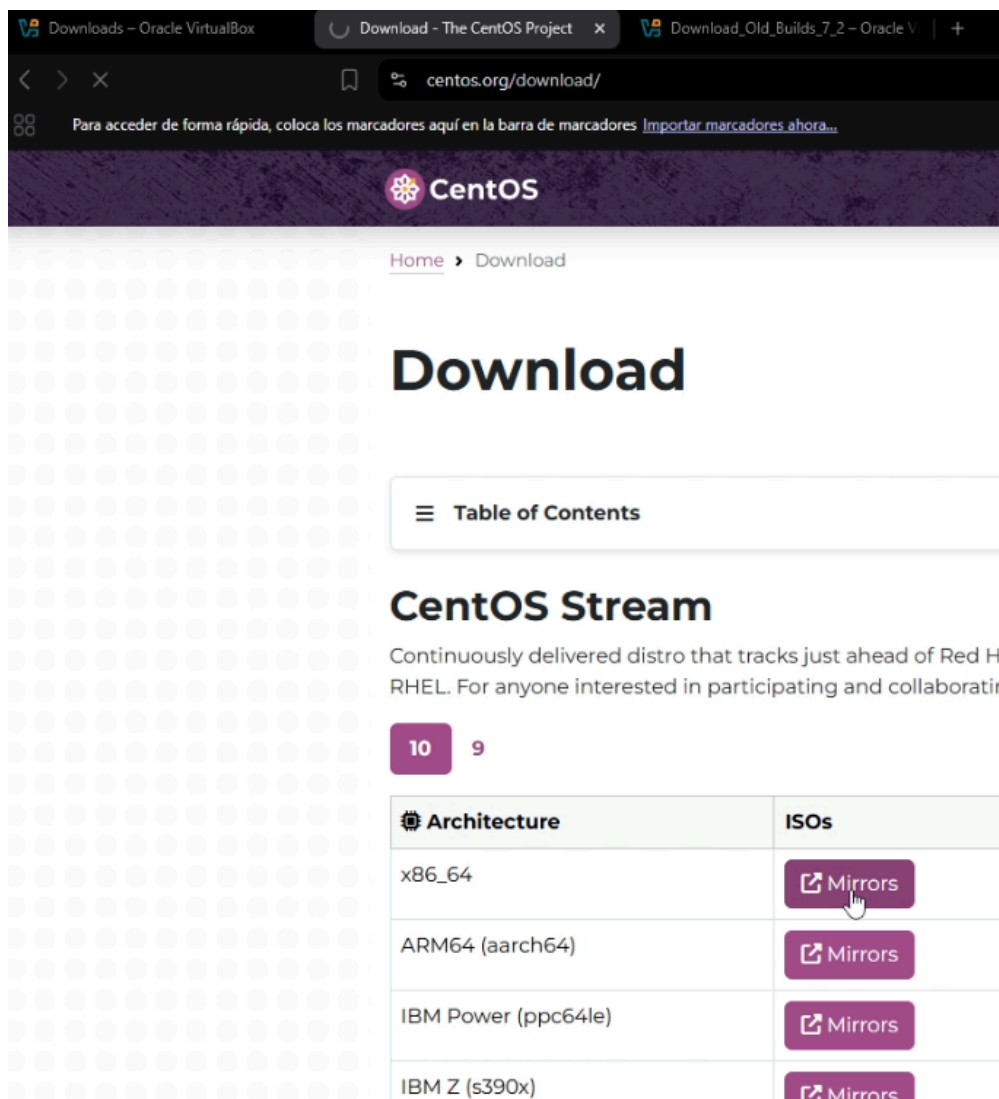
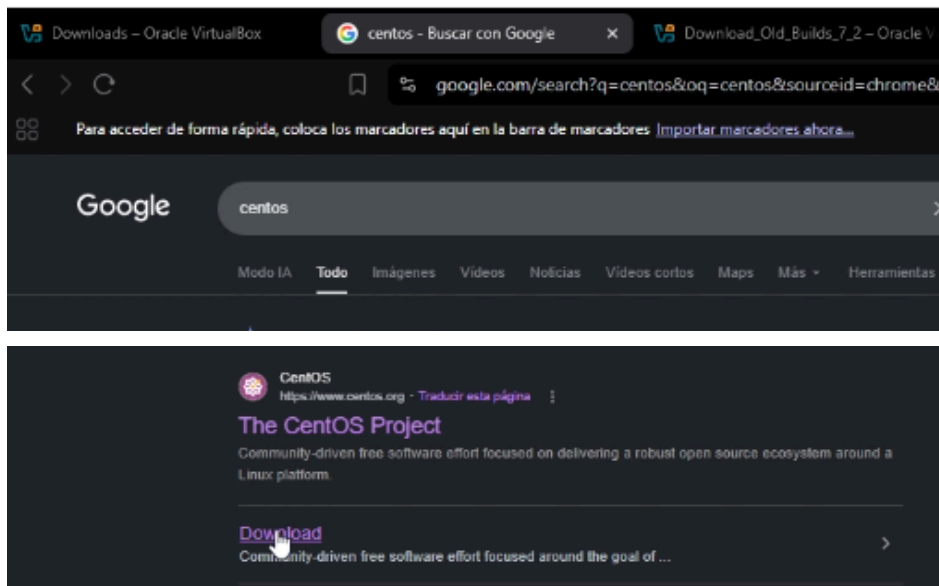
Previous

Install

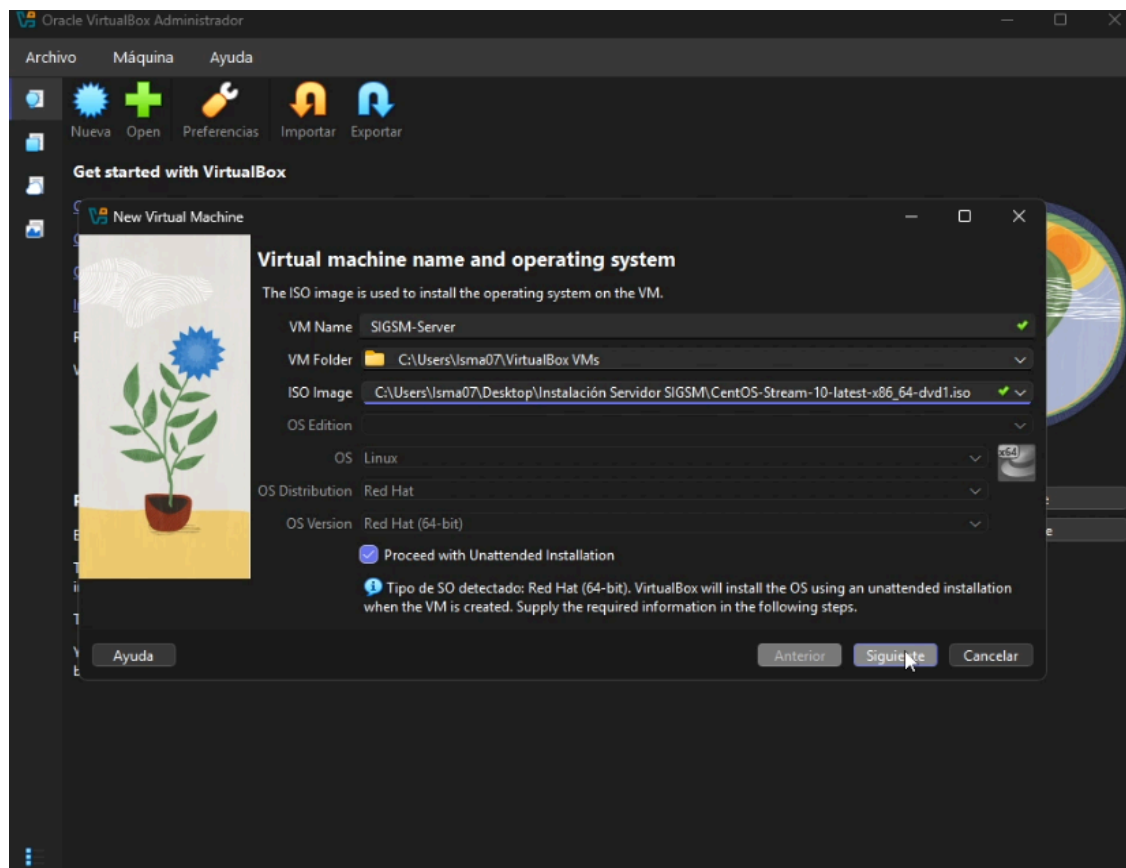
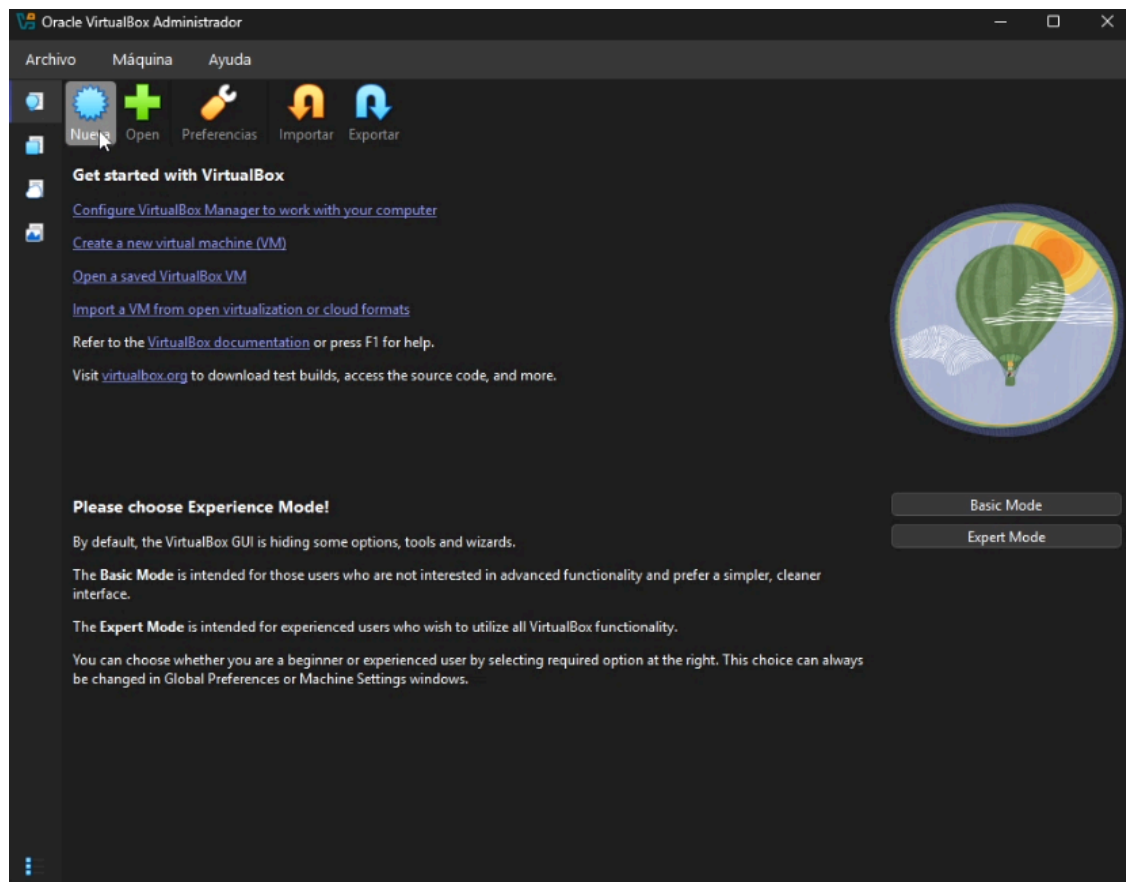
Cancel



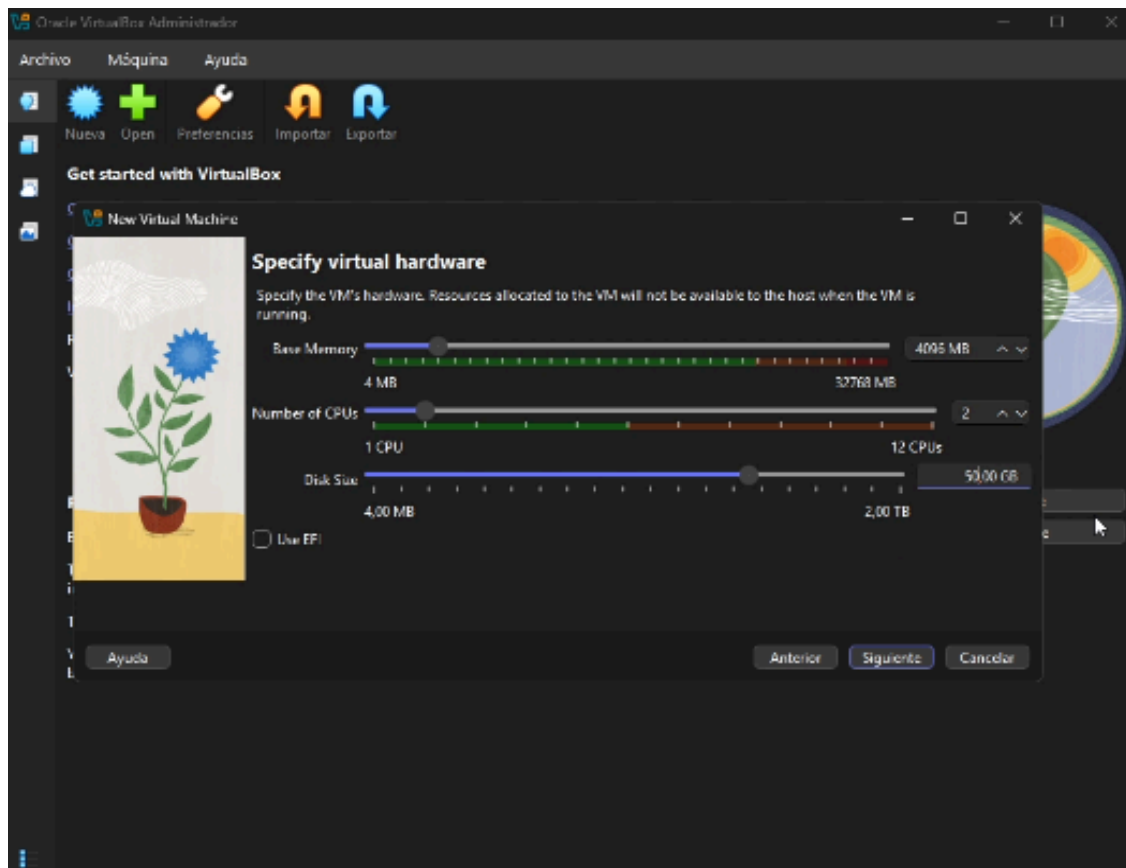
3) Descargar CentOS desde la web oficial



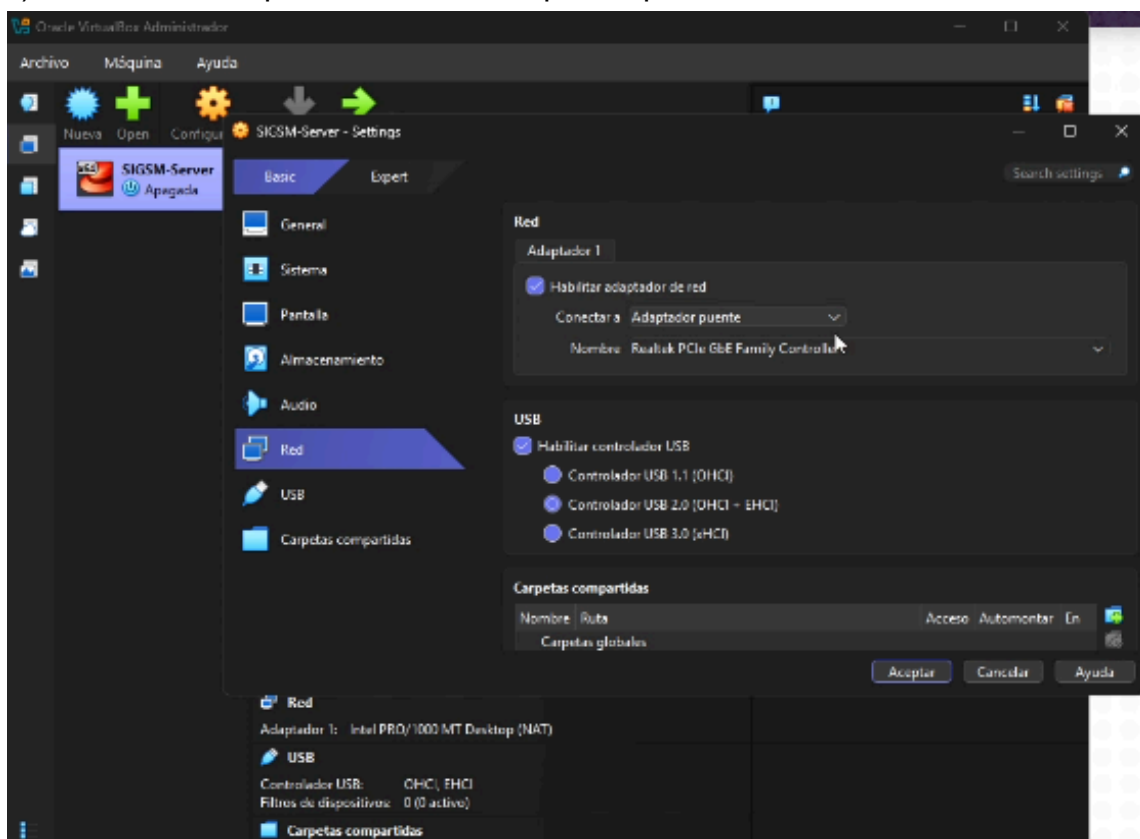
4) Crear una nueva máquina virtual en Virtualbox



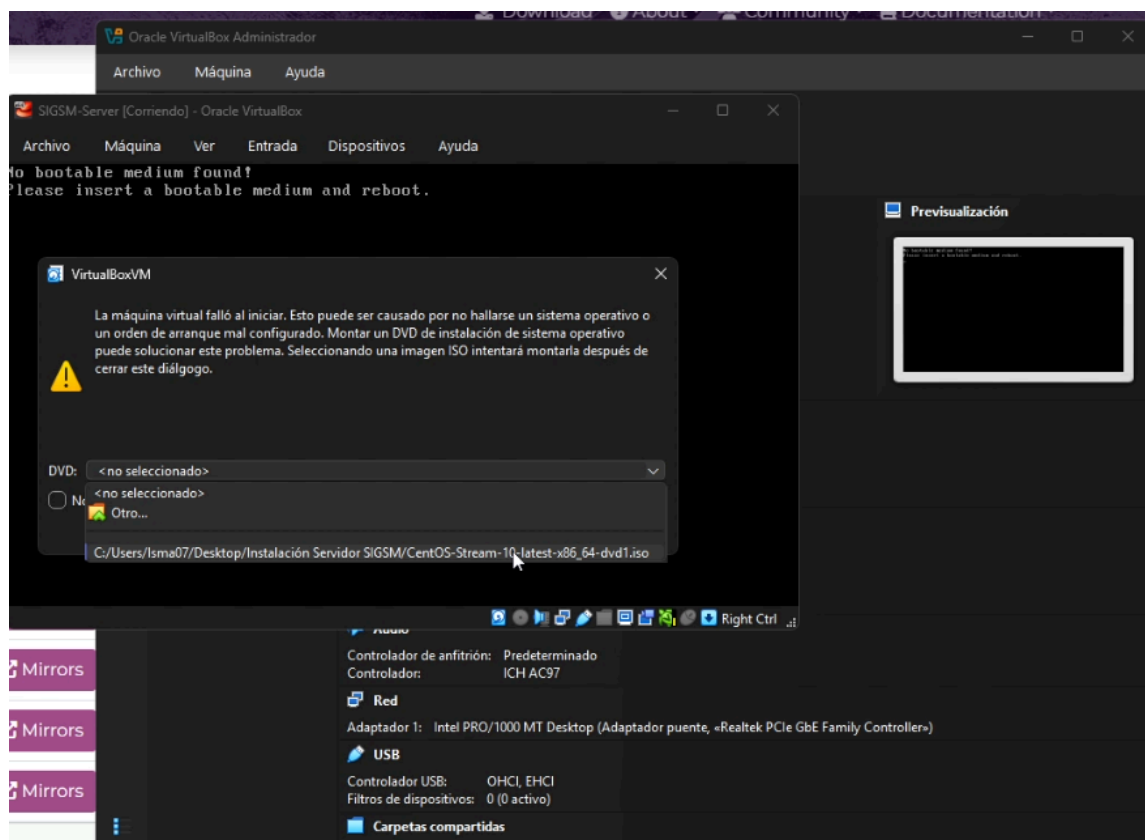
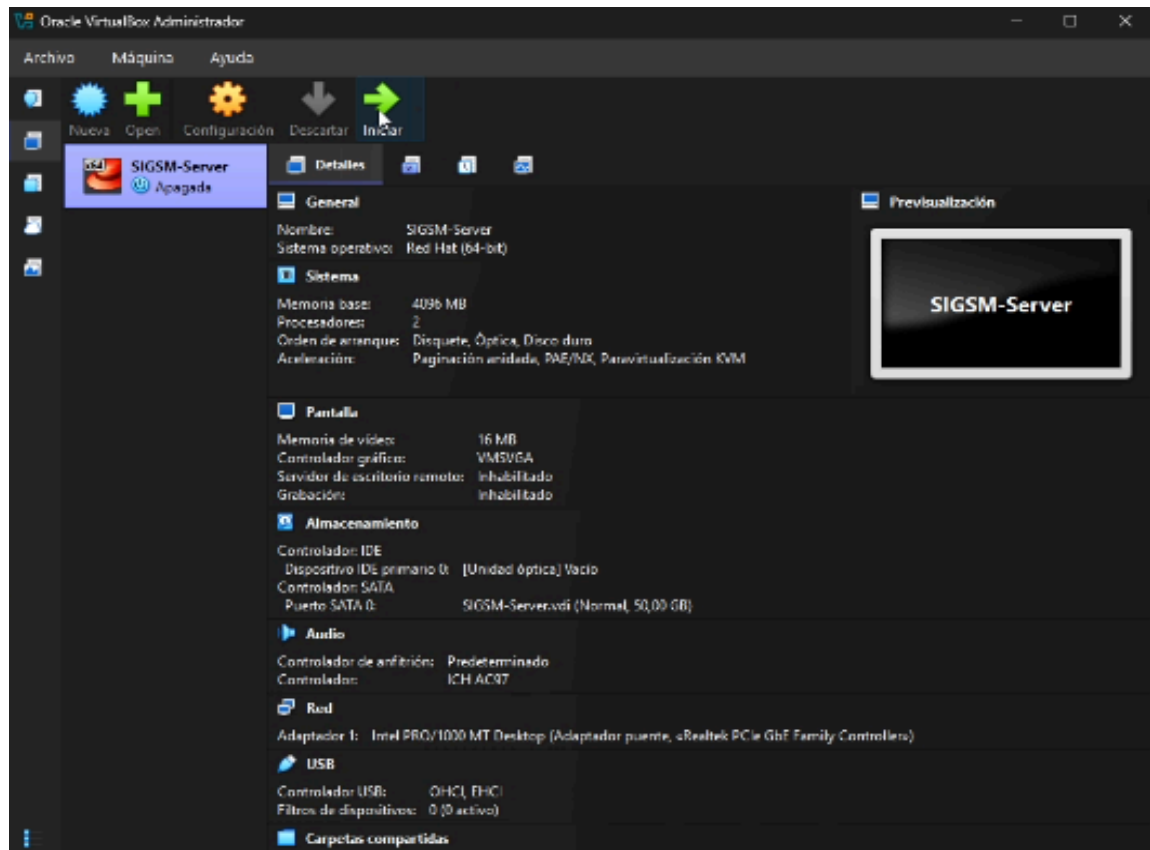
5) Configuración de sistema y hardware del servidor



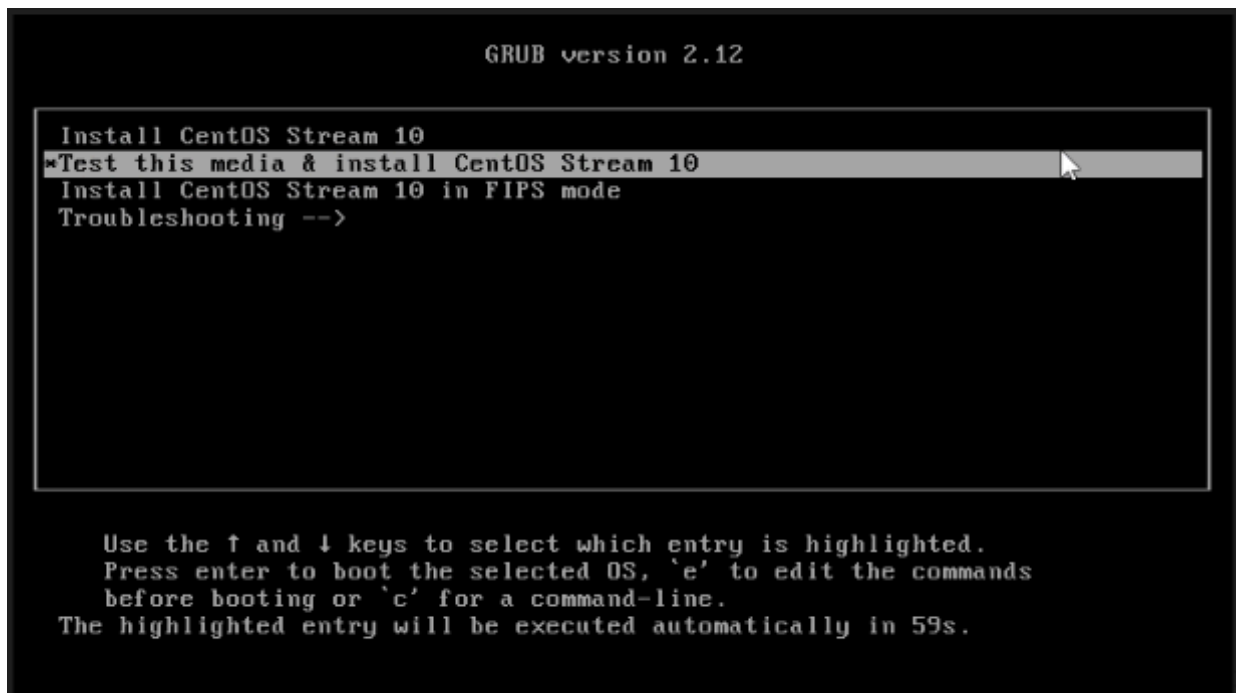
6) Cambiar el adaptador de red a adaptador puente



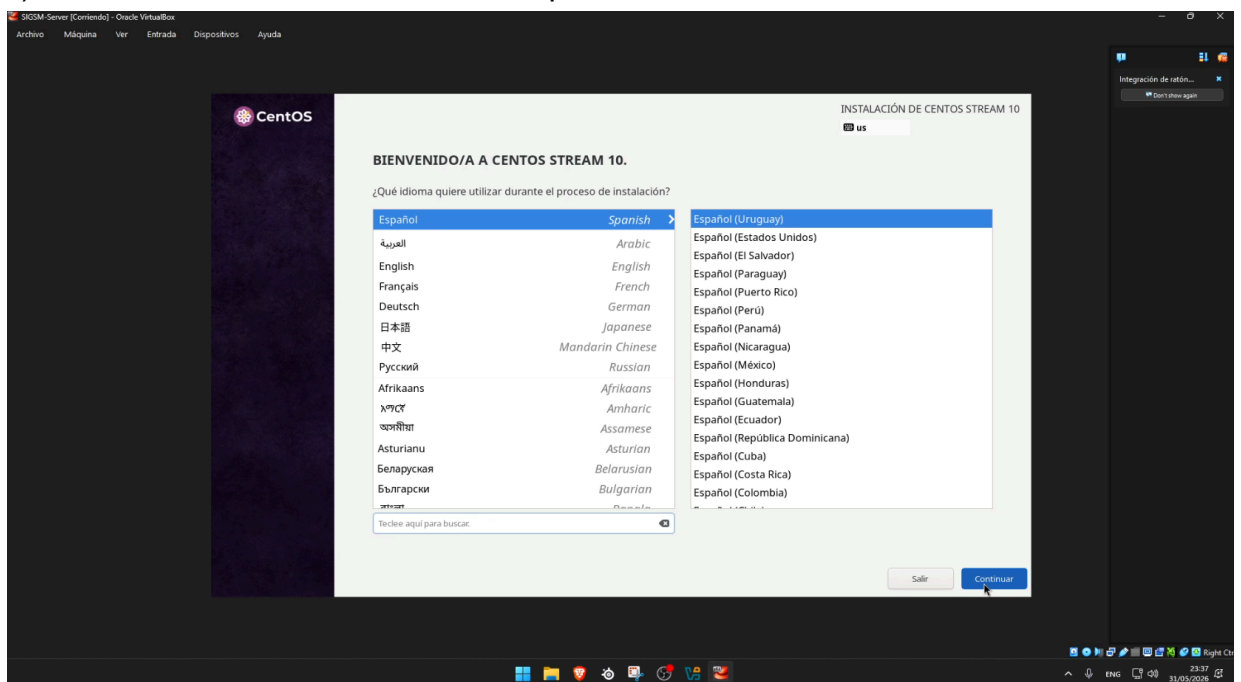
7) Iniciar la máquina virtual con CentOS 10



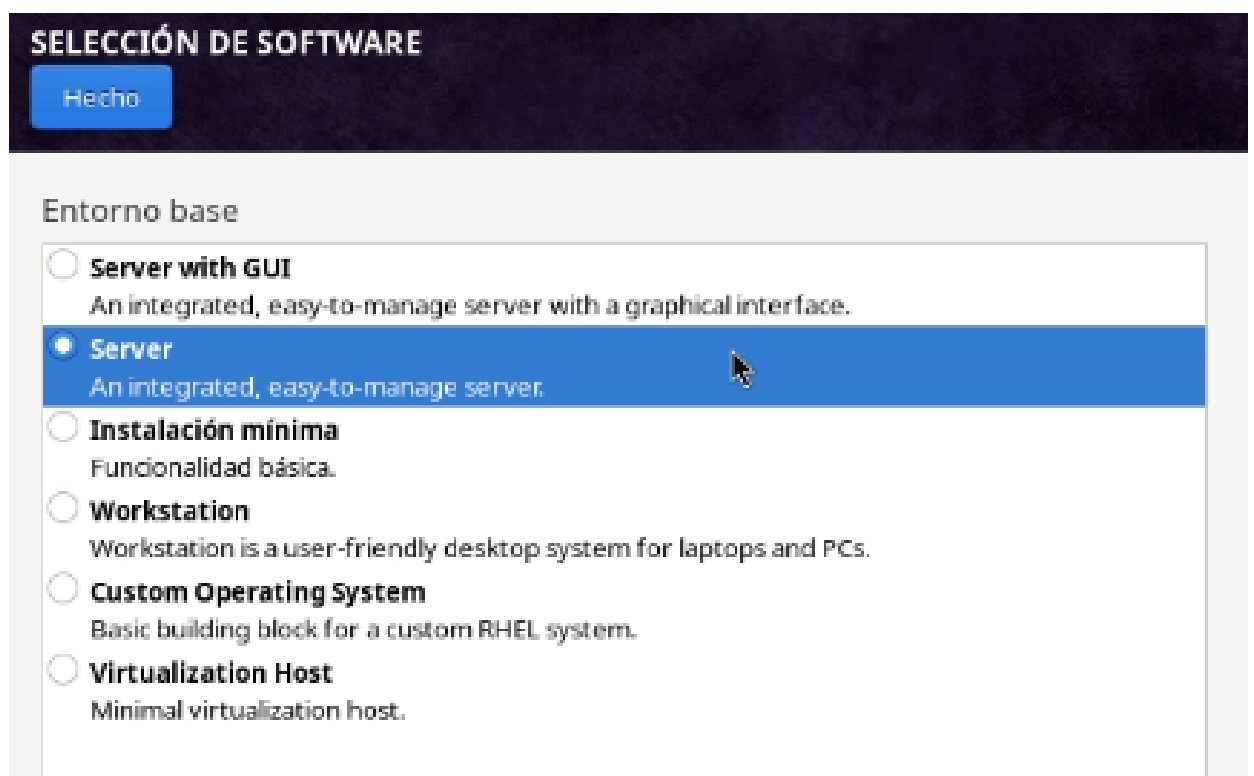
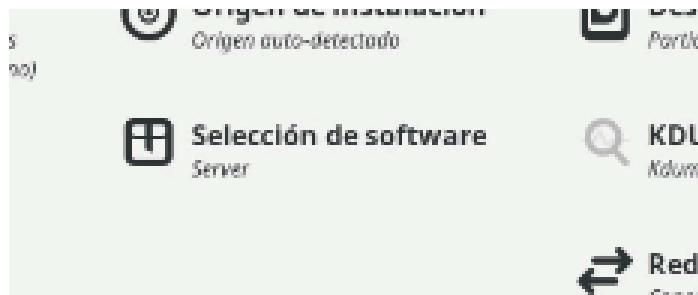
8) Comenzar la instalación de CentOS 10



9) Seleccionar el idioma del sistema operativo



10) Seleccionar Server como entorno base (sin GUI)



11) Activar la cuenta root

CUENTA DE ROOT

Hecho

La cuenta root se utiliza para administrar el sistema.

El usuario root (también conocido como superusuario) tiene acceso completo a todo el sistema. Por esta razón, es mejor iniciar sesión en este sistema como usuario root solo para realizar el mantenimiento o la administración del sistema.

☐ Desactivar la cuenta de root

Desactivar la cuenta de root bloqueará la cuenta y desactivará el acceso remoto con la cuenta de root. Esto evitará el acceso administrativo involuntario al sistema.

☒ Activar la cuenta de root

Habilitar la cuenta de root le permitirá establecer una contraseña de root y, opcionalmente, habilitar el acceso remoto a la cuenta de root en este sistema.

Contraseña administrativa:

Longitud insuficiente

Confirmar:

☒ Permitir el acceso SSH de root con contraseña

12) Click en comenzar instalación

CentOS

RESUMEN DE LA INSTALACIÓN

INSTALACIÓN DE CENTOS STREAM 10

us (altgr-i...)

REGIONALIZACIÓN	SOFTWARE	SISTEMA
Teclado Inglés (internacional con teclas muertas por Alt... latinoamericano)	Origen de instalación Origen auto-detectado	Destino de la instalación Particionado automático seleccionado
Soporte de idiomas Español (Uruguay)	Selección de software Server	KDUMP Kdump está habilitado
Fecha y hora Huso horario: América/ Montevideo		Red y nombre de equipo Conectado: enp0s3
AJUSTES DE USUARIO		
Cuenta de root Contraseña de root establecido		
Creación de usuario No se creará ningún usuario		

Salir Comenzar instalación

No tocaremos sus discos hasta que haga clic en "Comenzar la instalación".



13) Comando para verificar conexión a internet

```
SIGSM-Server [Corriendo] - Oracle VirtualBox

root@192:~# ping 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data:
64 bytes from 1.1.1.1: icmp_seq=1 ttl=54 time=11.6 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=54 time=13.0 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=54 time=11.6 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=54 time=11.7 ms
64 bytes from 1.1.1.1: icmp_seq=5 ttl=54 time=12.0 ms
64 bytes from 1.1.1.1: icmp_seq=6 ttl=54 time=11.6 ms
64 bytes from 1.1.1.1: icmp_seq=7 ttl=54 time=11.7 ms
64 bytes from 1.1.1.1: icmp_seq=8 ttl=54 time=12.0 ms
64 bytes from 1.1.1.1: icmp_seq=9 ttl=54 time=11.8 ms
64 bytes from 1.1.1.1: icmp_seq=10 ttl=54 time=12.0 ms
64 bytes from 1.1.1.1: icmp_seq=11 ttl=54 time=11.6 ms
64 bytes from 1.1.1.1: icmp_seq=12 ttl=54 time=12.3 ms
64 bytes from 1.1.1.1: icmp_seq=13 ttl=54 time=11.7 ms
^Z
[1]+  Detenido                  ping 1.1.1.1
root@192:~#
```

14) Para llevar a cabo las actualizaciones necesarias se utiliza el gestor de paquetes estándar de CentOS el cuál es DNF, ejecutando el comando `dnf update` periódicamente, de tal manera que el sistema se mantenga seguro y con las últimas novedades.

```
64 bytes from 1.1.1.1: icmp_seq=10 ttl=54 time=12.0 ms
64 bytes from 1.1.1.1: icmp_seq=11 ttl=54 time=11.6 ms
64 bytes from 1.1.1.1: icmp_seq=12 ttl=54 time=12.3 ms
64 bytes from 1.1.1.1: icmp_seq=13 ttl=54 time=11.7 ms
^Z
[1]+  Detenido                  ping 1.1.1.1
root@192:~# dnf update
CentOS Stream 10 - BaseOS
CentOS Stream 10 - AppStream          99% [-----
```

```
vim-data                                noarch
vim-enhanced                           x86_64
vim-filesystem                         noarch
vim-minimal                           x86_64
xxd                                    x86_64
Instalando dependencias:
kernel-core                           x86_64
kernel-modules                        x86_64
kernel-modules-core                   x86_64
kernel-modules-extra                  x86_64

Resumen de la transacción
=====
Instalar      5 Paquetes
Actualizar   35 Paquetes

Tamaño total de la descarga: 122 M
¿Está de acuerdo [s/N]?: s
Descargando paquetes:
CentOS Stream 10 - BaseOS             0% [
```

```
python3-pip-wheel-23.3.2-11.el10.noarch    selinux-policy-42.1.22-1.el10.noarch
util-linux-2.40.2-20.el10.x86_64           util-linux-core-2.40.2-20.el10.x86_64
vim-data-2:9.1.083-11.el10.noarch          vim-enhanced-2:9.1.083-11.el10.x86_64
vim-minimal-2:9.1.083-11.el10.x86_64      xxd-2:9.1.083-11.el10.x86_64
Instalado:
kernel-6.12.0-231.el10.x86_64              kernel-core-6.12.0-231.el10.x86_64
kernel-modules-core-6.12.0-231.el10.x86_64  kernel-modules-extra-6.12.0-231.e
¡Listo!
root@192:~#
```

15) Verificar la versión del sistema

```
iListo!
root@192:~# cat /etc/centos-release
CentOS Stream release 10 (Coughlan)
```

16) Verificar si se encuentra instalado OpenSSH y el demonio (proceso) disponible y ejecutándose por medio de systemctl (en caso de no estar disponible y/o ejecutándose, usar systemctl enable sshd y systemctl start sshd)

```
root@192:~# systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-05-31 23:44:32 -03; 1h 30min ago
 Invocation: d17e78e783464d27a0be652ff42d3bba
    Docs: man:sshd(8)
          man:sshd_config(5)
   Main PID: 1173 (sshd)
      Tasks: 1 (limit: 22991)
     Memory: 1.2M (peak: 1.4M)
        CPU: 13ms
    CGroup: /system.slice/ssh.service
            └─1173 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

may 31 23:44:32 localhost.localdomain systemd[1]: Starting sshd.service - OpenSSH server daemon...
may 31 23:44:32 localhost.localdomain sshd[1173]: Server listening on 0.0.0.0 port 22.
may 31 23:44:32 localhost.localdomain sshd[1173]: Server listening on :: port 22.
may 31 23:44:32 localhost.localdomain systemd[1]: Started sshd.service - OpenSSH server daemon.
root@192:~#
```

17) Verificar que el dnf-plugins-core esté instalado

```
root@192:~# dnf install -y dnf-plugins-core
Última comprobación de caducidad de metadatos hecha hace 1:06:26, el lun 01 jun 2026
El paquete dnf-plugins-core-4.7.0-10.el10.noarch ya está instalado.
Dependencias resueltas.
Nada por hacer.
iListo!
root@192:~#
```

18) Instalar los paquetes de docker y el plugin compose

```
root@192:~# sudo dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
Agregando repositorio de: https://download.docker.com/linux/rhel/docker-ce.repo
root@192:~# sudo dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Docker CE Stable - x86_64
Dependencias resueltas.
=====
Paquete                                Arquitectura  Versión
=====
Instalando:
containerd.io                          x86_64        2.2.4-1.el10
docker-buildx-plugin                   x86_64        0.34.1-1.el10
docker-ce                              x86_64        3:29.5.2-1.el10
docker-ce-cli                          x86_64        1:29.5.2-1.el10
docker-compose-plugin                  x86_64        5.1.4-1.el10
Instalando dependencias débiles:
docker-ce-rootless-extras              x86_64        29.5.2-1.el10
=====
Resumen de la transacción
=====
Instalar 6 Paquetes

Tamaño total de la descarga: 100 M
Tamaño instalado: 378 M
¿Está de acuerdo [s/N]? : s
Descargando paquetes:
(1-3/6): docker-ce-29.5.2-1.el10.x86_64.rpm          4% [==-
```

```
Resumen de la transacción
=====
Instalar 6 Paquetes

Tamaño total de la descarga: 100 M
Tamaño instalado: 378 M
¿Está de acuerdo [s/N]? : s
Descargando paquetes:
(1/6): docker-buildx-plugin-0.34.1-1.el10.x86_64.rpm
(2/6): docker-ce-29.5.2-1.el10.x86_64.rpm
[MIRROR] docker-ce-rootless-extras-29.5.2-1.el10.x86_64.rpm: Curl error (28): Timeout was reached
ckages/docker-ce-rootless-extras-29.5.2-1.el10.x86_64.rpm [Operation too slow. Less than 1000 bytes
(3/6): docker-ce-cli-29.5.2-1.el10.x86_64.rpm
(4/6): docker-ce-rootless-extras-29.5.2-1.el10.x86_64.rpm
[MIRROR] docker-compose-plugin-5.1.4-1.el10.x86_64.rpm: Curl error (28): Timeout was reached
s/docker-compose-plugin-5.1.4-1.el10.x86_64.rpm [Operation too slow. Less than 1000 bytes
(5/6): containerd.io-2.2.4-1.el10.x86_64.rpm
(6/6): docker-compose-plugin-5.1.4-1.el10.x86_64.rpm          95% [=====
```

19) Verificamos el estado de docker

```
root@192:~# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-06-01 01:54:41 -03; 2min 47s ago
 Invocation: 22645d7c9ce242b281c19c817a3ce25c
 TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 21672 (dockerd)
     Tasks: 11
    Memory: 37M (peak: 39.8M)
       CPU: 409ms
    CGroup: /system.slice/docker.service
            └─21672 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

jun 01 01:54:41 192.168.1.92 dockerd[21672]: time="2026-06-01T01:54:41.394226156-03:00" level=info
jun 01 01:54:41 192.168.1.92 dockerd[21672]: time="2026-06-01T01:54:41.394348431-03:00" level=info
jun 01 01:54:41 192.168.1.92 dockerd[21672]: time="2026-06-01T01:54:41.407543955-03:00" level=warn
jun 01 01:54:41 192.168.1.92 dockerd[21672]: time="2026-06-01T01:54:41.415289959-03:00" level=info
jun 01 01:54:41 192.168.1.92 dockerd[21672]: time="2026-06-01T01:54:41.418220009-03:00" level=info
jun 01 01:54:41 192.168.1.92 dockerd[21672]: time="2026-06-01T01:54:41.418291948-03:00" level=info
jun 01 01:54:41 192.168.1.92 systemd[1]: Started docker.service - Docker Application Container Eng
jun 01 01:55:01 192.168.1.92 dockerd[21672]: time="2026-06-01T01:55:01.348600295-03:00" level=info
jun 01 01:55:01 192.168.1.92 dockerd[21672]: time="2026-06-01T01:55:01.623434269-03:00" level=info
jun 01 01:55:01 192.168.1.92 dockerd[21672]: time="2026-06-01T01:55:01.690450168-03:00" level=info
lines 1-23/23 (END)
```

20) Habilitamos docker y corremos la imagen hello-world

```
¡Listo!
root@192:~# systemctl enable --now docker
Created symlink '/etc/systemd/system/multi-
[ 7816.012476] Warning: Unmaintained driver
[ 7816.015175] Warning: Unmaintained driver
[ 7816.191515] Warning: Unmaintained driver
[ 7816.195097] Warning: Unmaintained driver
root@192:~# docker run hello-world
Unable to find image 'hello-world:latest' l
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:0e760fd9fbc48ba8041e7c6db999b
Status: Downloaded newer image for hello-w

Hello from Docker!
This message shows that your installation a

To generate this message, Docker took the f
1. The Docker client contacted the Docker
```

21) Verificar comando ip addr

```
root@192:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:00:27:a3:a6:46 brd ff:ff:ff:ff:ff:ff
    altname enx000027a3a646
    inet 192.168.1.92/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s3
        valid_lft 78387sec preferred_lft 78387sec
    inet6 2000:a4:96c:700:a00:27ff:fea3:a646/64 scope global deprecated dynamic noprefixroute
        valid_lft 1293sec preferred_lft 0sec
    inet6 2000:a4:8f1:8d00:a00:27ff:fea3:a646/64 scope global deprecated dynamic noprefixroute
        valid_lft 5161sec preferred_lft 0sec
    inet6 2000:a4:e37:7500:a00:27ff:fea3:a646/64 scope global dynamic noprefixroute
        valid_lft 41876sec preferred_lft 41876sec
    inet6 fe80::a00:27ff:fea3:a646/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether e6:e4:ca:cf:36:2f brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::e4e4:caff:febf:362f/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

22) Verificar la versión de git (ya preinstalada en Centos 10)

```
SIGSM-Server [Corriendo] - Oracle VirtualBox

Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

CentOS Stream 10 (Coughlan)
Kernel 6.12.0-231.el10.x86_64 on x86_64

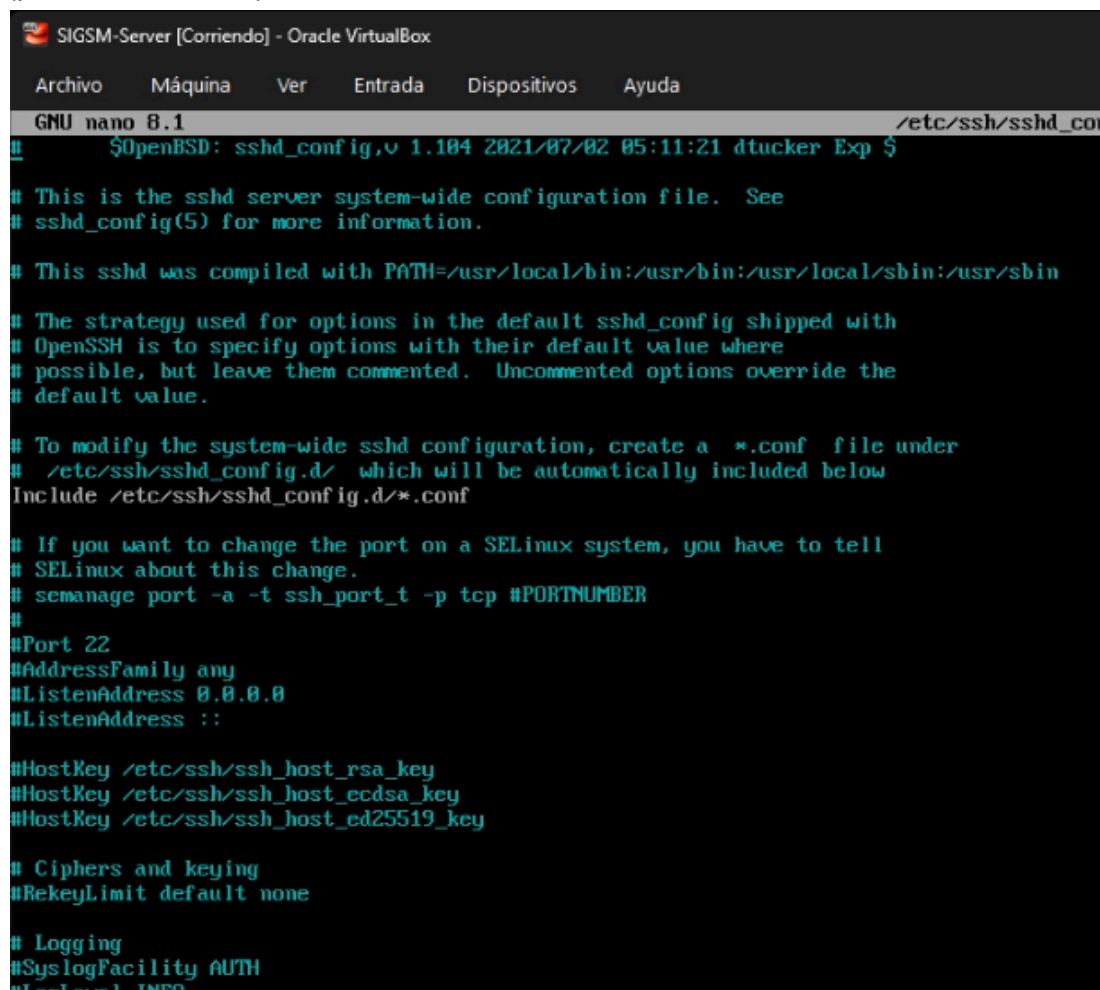
Web console: https://192.168.1.92:9090/ or https://192.168.1.92:9090/

192 login: root
Password:
Last login: Tue Jun  9 20:40:51 from 192.168.1.4
root@192:~# git --version
git version 2.52.0
root@192:~# _
```

23) Verificar servicios con firewall-cmd

```
root@192:~# firewall-cmd --list-services
cockpit dhcpv6-client ssh
```

24) Con este comando se puede visualizar y modificar el archivo sshd, en el cuál por ejemplo se encuentra el puerto perteneciente al servicio (por defecto el 22)



```
SIGSM-Server [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
GNU nano 8.1 /etc/ssh/sshd_config
# $OpenBSD: sshd_config,v 1.104 2021/07/02 05:11:21 dtucker Exp $

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

# To modify the system-wide sshd configuration, create a *.conf file under
# /etc/ssh/sshd_config.d/ which will be automatically included below
Include /etc/ssh/sshd_config.d/*.conf

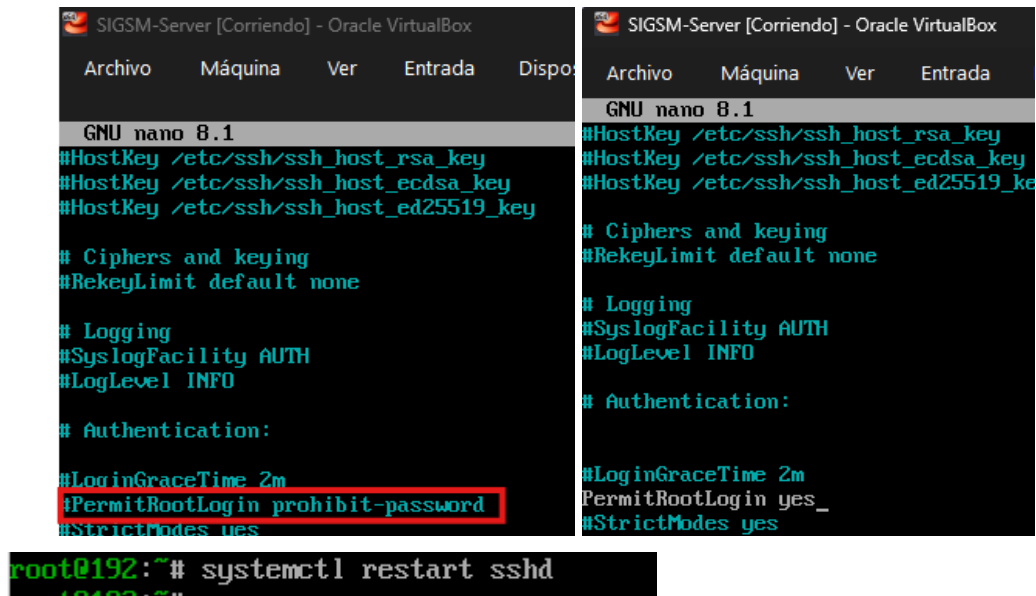
# If you want to change the port on a SELinux system, you have to tell
# SELinux about this change.
# semanage port -a -t ssh_port_t -p tcp #PORTNUMBER
#
#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO
```


25) Bajando se encuentra una línea que dice `#PermitRootLogin prohibit-password`, para hacer la comprobación de conexión por ssh la descomentamos y cambiamos a `PermitRootLogin` yes, luego salimos de ssh y ejecutamos `systemctl restart sshd` y pasamos al siguiente paso.



```
SIGSM-Server [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Disposición

GNU nano 8.1
#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:
#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:
#LoginGraceTime 2m
PermitRootLogin yes_
#StrictModes yes

root@192:~# systemctl restart sshd
```

26) Además de las comprobaciones en pasos anteriores como pruebas de una correcta instalación, se conecta por medio de ssh desde la terminal de windows a la máquina virtual, con el comando `ip addr` visto anteriormente se ve la ip del servidor dentro de la interfaz `enp0s3` donde dice `inet`, luego en cmd se pone `ssh [usuario]@[ ip ]`.

```
SIGSM-Server [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

CentOS Stream 10 (Coughlan)
Kernel 6.12.0-231.el10.x86_64 on x86_64

Web console: https://192.168.1.92:9090/ or https://192.168.1.92:9090/

192 login: root
Password:
Last login: Mon Jun  1 02:01:44 on tty1
root@192:~# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
    link/ether 08:00:27:a3:a6:46 brd ff:ff:ff:ff:ff:ff
    altname enx080027a3a646
    inet 192.168.1.92/24 brd 192.168.1.255 scope global dyna
        valid_lft 86325sec preferred_lft 86325sec
    inet6 2800:a4:c1f4:2e00:a00:27ff:fea3:a646/64 scope glob
        valid_lft 25517sec preferred_lft 25517sec
    inet6 fe80::a00:27ff:fea3:a646/64 scope link noprefixrou
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdi
    link/ether 0a:11:13:f8:39:d7 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docke
        valid_lft forever preferred_lft forever
root@192:~#
```

27) Si ingresamos correctamente los datos podemos observar que la conexión fue exitosa, y estamos ya dentro de la máquina virtual, para comprobarlo podemos por ejemplo apagarla con poweroff y veremos que la máquina virtual SIGSM-Server se apaga, confirmando la conexión ssh.

```
root@192:~
Microsoft Windows [Versión 10.0.26200.7462]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Windows\System32>ssh root@192.168.1.92
The authenticity of host '192.168.1.92 (192.168.1.92)' can't be established.
ED25519 key fingerprint is SHA256:PCNsRlGcWVNSH+CvpwkfIU2sRTSJiCrCNXaxNTzd4tg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.92' (ED25519) to the list of known hosts.
root@192.168.1.92's password:
Connection closed by 192.168.1.92 port 22

C:\Windows\System32>ssh root@192.168.1.92
kex_exchange_identification: read: Connection reset
Connection reset by 192.168.1.92 port 22

C:\Windows\System32>ssh root@192.168.1.92
root@192.168.1.92's password:
Web console: https://192.168.1.92:9090/ or https://192.168.1.92:9090/

Last login: Tue Jun  9 19:53:18 2026
root@192:~#
```

```
Connection reset by 192.168.1.92 port 22
C:\Windows\System32>ssh root@192.168.1.92
root@192.168.1.92's password:
Web console: https://192.168.1.92:9090/ or https://192.168.1.92:9090/
Last login: Tue Jun  9 19:53:18 2026
root@192:~# poweroff
root@192:~# client_loop: send disconnect: Connection reset
C:\Windows\System32>
```

Incidentes de seguridad sobre los paquetes a instalar

CVE-2015-6249: En 2015 se encontró una falla en OpenSSH por cómo manejaba la autenticación PAM cuando utilizaba la separación de privilegios. Un atacante con credenciales válidas en el sistema capaz de comprometer completamente un proceso de pre autenticación sin privilegios utilizando una falla diferente podría usar esta falla para autenticarse como un usuario diferente.

CVE-2014-5282: En 2014 se encontró una falla de seguridad ya que Docker 1.0.0 utiliza permisos legibles y escribibles en el socket de administración, lo cual permite a los usuarios locales obtener privilegios a través de vectores no especificados.

CVE-2014-5277: Docker anterior a 1.3.1 y docker-py anterior a 0.5.3 vuelven a HTTP cuando falla la conexión HTTPS al registro, lo que permite a los atacantes intermediarios realizar ataques de degradación y obtener datos de autenticación e imágenes aprovechando una posición de red entre el cliente y el registro para bloquear el tráfico HTTPS.

CVE-2015-7545: Se trata de una falla descubierta en 2015 en cuanto a la forma en que git-remote-ext procesaba ciertas URL, lo cuál si un usuario clonaba automáticamente submódulos de repositorios, de los cuáles no se verifica su seguridad, un atacante le podría introducir código malicioso e inyectar comandos en la URL de dichos submódulos, ejecutando código arbitrario en el sistema de usuario.

Vulnerabilidades en versiones anteriores

En cuanto a CentOS ya que está basado en RedHat Enterprise Linux (RHEL), se puede mencionar que en 2024 hubo una vulnerabilidad que se utilizaba activamente en campañas de ransomware, nombrada como CVE-2024-1086. Se trata de una falla en el subsistema netfilter en el kernel de linux, más específicamente en el componente llamado nf_tables, el cual permitía a los atacantes conseguir acceso local y permisos de root en el sistema. La falla es originaria de código que existe desde 2014, siendo divulgada y parcheada en 2024.

Ciertas vulnerabilidades como una reciente (CVE-2026-43824) del primero de mayo de 2026 no utilizan un paquete base del sistema operativo. Ello implica que podría tener que arreglarse manualmente, por lo tanto se revisarán las vulnerabilidades recientes de los paquetes que vayan a instalarse para así evitar riesgos. De todas formas el componente de esta vulnerabilidad mencionada es argo cd, y no va a ser utilizado, ya que resulta bastante avanzado y es más razonable utilizar Docker Compose. (Red Hat Customer Portal "Security Updates")